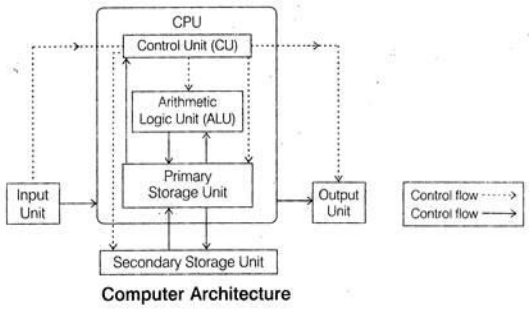
Chapter I
Introduction to Computer Architecture

ARCHITECTURE OF COMPUTER SYSTEM

Computer is an electronic machine that makes performing any task very easy. In computer, the CPU executes each instruction provided to it, in a series of steps; this series of steps is called Machine Cycle, and is repeated for each instruction. One machine cycle involves fetching of instruction, decoding the instruction, transferring the data, executing the instruction.
Computer system has five basic units that help the computer to perform operations, which are given below:

- Input Unit
- Output Unit
- Storage Unit
- Arithmetic Logic Unit
- Control Unit



Computer Architecture

Input Unit: It connects the external environment with internal computer system. It provides data and instructions to the computer system. Commonly used input devices are keyboard, mouse, and magnetic tape etc. Input unit performs following tasks:

- Accept the data and instructions from the outside environment.
- Convert it into machine language.
- Supply the converted data to computer system.

Output Unit: It connects the internal system of a computer to the external environment. It provides the results of any computation, or instructions to the outside world. Some output devices are printers, monitor etc.
Storage Unit: This unit holds the data and instructions. It also stores the intermediate results before these are sent to the output devices. It also stores the data for later use.

The storage unit of a computer system can be divided into two categories:

- Primary Storage: This memory is used to store the data which is being currently executed. It is used for temporary storage of data. The data is lost, when the computer is switched off. RAM is used as primary storage memory.
- Secondary Storage: The secondary memory is slower and cheaper than primary memory. It is used for permanent storage of data. Commonly used secondary memory devices are hard disk, CD etc.

Arithmetic Logical Unit: All the calculations are performed in ALU of the computer system. The ALU can perform basic operations such as addition, subtraction, division, multiplication etc. Whenever calculations are required, the control unit transfers the data from storage unit to ALU. When the operations are done, the result is transferred back to the storage unit.

Control Unit: It controls all other units of the computer. It controls the flow of data and instructions to and from the storage unit to ALU. Thus it is also known as central nervous system of the computer.

CPU: It is Central Processing Unit of the computer. The control unit and ALU are together known as CPU. CPU is the brain of computer system. It performs following tasks:

- It performs all operations.
- It takes all decisions.
- It controls all the units of computer.

LAYERS OF ABSTRACTION IN COMPUTER SYSTEM

- In computing, an abstraction layer or abstraction level is a way of hiding the working details of a subsystem, allowing the separation of concerns to facilitate interoperability and platform independence.
- Every layer can exist without the layers above it, and requires the layers below it to function.

- Computer System is divided into two functional entities. *Hardware* and *Software* are two functional entities of computer system.
- Operating system is the link between hardware and software.
- There are a certain layers in computer system through which a process goes to perform a task.

Following are the different layers of abstraction in computer system:

- **Problem Statement**–Problem Statement is stated using natural language. It may be ambiguous or imprecise. It is basically the user's requirement from the system.
- **Algorithm** –Algorithm is the step by step procedure to perform a specific task. It is guaranteed to finish. It has definiteness, effective computability and finiteness.
- **Program** –Program expresses the algorithm using a computer language such as high level language and low level language. User writes the code for their problem statement.
- **Instruction Set Architecture** –Instruction set architecture specifies the set of instructions the computer can perform using data types and addressing modes.
- **Micro-architecture**–Micro-architecture is the detailed organization of a processor implementation.
- **Logic Circuits**–Logic circuits combine basic operations to realize micro-architecture.
- **Device** – Device has the properties of materials and manufacturability.

8 GREAT IDEAS IN COMPUTER ARCHITECTURE

Application of the following great ideas has accounted for much of the tremendous growth in computing capabilities over the past 50 years.

- Design for Moore's law
- Use abstraction to simplify design
- Make the common case fast
- Performance via parallelism

- Performance via pipelining
- Performance via prediction
- Hierarchy of memories
- Dependability via redundancy

Design for Moore's Law:

- Gordon Moore, one of the founders of Intel made a prediction in 1965 that integrated circuit resources would double every 18–24 months.
- This prediction has held approximately true for the past 50 years. It is now known as Moore's Law.
- When computer architects are designing or upgrading the design of a processor they must anticipate where the competition will be in 3-5 years when the new processor reaches the market.
- Targeting the design to be just a little bit better than today's competition is not good enough.

Use Abstraction to Simplify Design:

- Abstraction uses multiple levels with each level hiding the details of levels below it.
- For example: The instruction set of a processor hides the details of the activities involved in executing an instruction. High-level languages hide the details of the sequence of instructions need to accomplish a task. Operating systems hide the details involved in handling input and output devices.

Make the Common Case Fast:

- The most significant improvements in computer performance come from improvements to the common case areas where the current design is spending the most time.
- This idea is sometimes called Amdahl's law, though it is preferable to use that term to refer to a mathematical law for analyzing improvements. The mathematical law also is closely related to the law of diminishing returns.

Performance via Parallelism:

- Doing different parts of a task in parallel accomplishes the task in less time than doing them sequentially.
- A processor engages in several activities in the execution of an instruction.
- It runs faster if it can do these activities in parallel.

Performance via Pipelining:

- This idea is an extension of the idea of parallelism. It is essentially handling the activities involved in instruction execution as an assembly line.
- As soon as the first activity of an instruction is done you move it to the second activity and start the first activity of a new instruction.
- These results in executing more instructions per unit time compared to waiting for all activities of the first instruction to complete before starting the second instruction.

Performance via Prediction:

- A conditional branch is a type of instruction determines the next instruction to be executed based on a condition test.
- Conditional branches are essential for implementing high-level language if statements and loops.
- Unfortunately, conditional branches interfere with the smooth operation of a pipeline — the processor does not know where to fetch the next instruction until after the condition has been tested.
- Many modern processors reduce the impact of branches with speculative execution: make an informed guess about the outcome of the condition test and start executing the indicated instruction.
- Performance is improved if the guesses are reasonably accurate and the penalty of wrong guesses is not too severe.

Hierarchy of Memories:

- The principle of locality states that memory that has been accessed recently is likely to be accessed again in the near future.

- That is, accessing recently accessed data is a common case for memory accesses.
- To make this common case faster you need a cache-a small high-speed memory designed to hold recently accessed data.
- Modern processors use as many as 3 levels of caches. This is motivated by the large difference in speed between processors and memory.

Dependability via Redundancy:

- One of the most important ideas in data storage is the Redundant Array of Inexpensive Disks (RAID) concept.
- In most versions of RAID, data is stored redundantly on multiple disks.
- The redundancy insures that if one disk fails the data can be recovered from other disks.

FIVE KEY COMPONENTS OF A COMPUTER

The five classic components of a computer are briefly described below. Each component is discussed in more detail in its own section. The operation of the processor is best understood in terms of these components

- Datapath - manipulates the data coming through the processor. It also provides a small amount of temporary data storage.
- Control - generates control signals that direct the operation of memory and the datapath.
- Memory - holds instructions and most of the data for currently executing programs.
- Input - external devices such as keyboards, mice, disks, and networks that provide input to the processor.
- Output - external devices such as displays, printers, disks, and networks that receive data from the processor

Datapath

The datapath manipulates the data coming through the processor. It also provides a small amount of temporary data storage.

The datapath consists of the following components.

- programmable registers - small units of data storage that are directly visible to assembly language programmers. They can be used like simple variables in a high-level program.
- the program counter (PC) - holds the address for fetching instructions.
- multiplexers have control inputs coming from control. They are used for routing data through the datapath.
- processing elements - compute new data values from old data values. In simple processors the major processing elements are grouped into an Arithmetic-Logic Unit (ALU).
- special-purpose registers - hold data that is needed for processor operation but is not directly visible to assembly language programmers.

Control

Control generates control signals that direct the operation of memory and the datapath. The control signals do the following.

- Tell memory to send or receive data.
- Tell the ALU what operation to perform.
- Route data between different parts of the datapath.

Memory

Memory holds instructions and most of the data for currently executing programs.

The rest of the data is held in programmable registers, which can only hold a limited amount of data.

Input

Input is data coming into the processor from external input devices such as keyboards, mice, disks, and networks.

In modern processors, this data is placed in memory before entering the processor.

Input handling is largely under the control of operating system software.

Output

Output is data going from the processor to external output devices such as displays, printers, disks, and networks.

In modern processors, this data is placed in memory before leaving the processor.
Output handling is largely under the control of operating system software

Processor Operation

The processor executes a sequence of instructions that are located in memory.
Execution of each instruction involves at least the first three of the following activities.
The last four activities are required for some, but not all, instructions.
The activities are approximately in time order. However, some of the activities can be overlapped in time.

- Instruction fetch
- Program counter (PC) update
- Instruction decode
- Source operand fetch
- Arithmetic-logic unit (ALU) operation
- Memory access
- Register write

The program counter (PC) hold the address of the next instruction.

- For a simple processor, the arithmetic-logic unit (ALU) performs all arithmetic and logical operations.

The organization of the data path can be determined from these activities.
Where an activity requires selecting among different options depending on the instruction, there will be a multiplexer that selects the appropriate option as directed by a control signal.

COMPUTER ARCHITECTURE: INSTRUCTION CODES

While a Program, as we all know, is, a set of instructions that specify the operations, operands, and the sequence by which processing has to occur. An instruction code is a group of bits that tells the computer to perform a specific operation part.

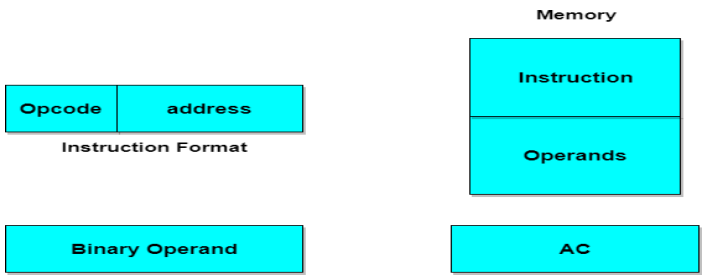
Instruction Code: Operation Code

The operation code of an instruction is a group of bits that define operations such as add, subtract, multiply, shift and compliment. The number of bits required for the operation code depends upon the total number of operations available on the computer. The operation code must consist of at least n bits for a given 2^n operations. The operation part of an instruction code specifies the operation to be performed.

Instruction Code: Register Part

The operation must be performed on the data stored in registers. An instruction code therefore specifies not only operations to be performed but also the registers where the operands (data) will be found as well as the registers where the result has to be stored.

Stored Program Organisation



The simplest way to organize a computer is to have Processor Register and instruction code with two parts. The *first part* specifies the operation to be performed and *second* specifies an address. The memory address tells where the operand in memory will be found.

Instructions are stored in one section of memory and data in another.

Computers with a single processor register are known as Accumulator (AC). The operation is performed with the memory operand and the content of AC.

Common Bus System

The basic computer has 8 registers, a memory unit and a control unit. Paths must be provided to transfer data from one register to another. An efficient method for transferring data in a system is to use a Common Bus System. The output of registers and memory are connected to the common bus.

Load(LD)

The lines from the common bus are connected to the inputs of each register and data inputs of memory. The particular register whose LD input is enabled receives the data from the bus during the next clock pulse transition.

Before studying about instruction formats lets first study about the operand address parts.

When the 2nd part of an instruction code specifies the operand, the instruction is said to have immediate operand. And when the 2nd part of the instruction code specifies the address of an operand, the instruction is said to have a direct address. And in indirect address, the 2nd part of instruction code, specifies the address of a memory word in which the address of the operand is found.

Computer Instructions

The basic computer has three instruction code formats. The Operation code (opcode) part of the instruction contains 3 bits and remaining 13 bits depends upon the operation code encountered.There are three types of formats:

Memory Reference Instruction

It uses 12 bits to specify the address and 1 bit to specify the addressing mode (I). I is equal to 0 for direct address and 1 for indirect address.

Register Reference Instruction

These instructions are recognized by the opcode 111 with a 0 in the left most bit of instruction. The other 12 bits specify the operation to be executed.

Input-Output Instruction

These instructions are recognized by the operation code 111 with a 1 in the left most bit of instruction. The remaining 12 bits are used to specify the input-output operation.

Format of Instruction

The format of an instruction is depicted in a rectangular box symbolizing the bits of an instruction. Basic fields of an instruction format are given below:

1. An operation code field that specifies the operation to be performed.

2. An address field that designates the memory address or register.

3. A mode field that specifies the way the operand of effective address is determined.

Computers may have instructions of different lengths containing varying number of addresses. The number of address field in the instruction format depends upon the internal organization of its registers.

INSTRUCTION SET ARCHITECTURE (ISA)

The Instruction Set Architecture (ISA) is the part of the processor that is visible to the programmer or compiler writer. The ISA serves as the boundary between software and hardware. We will briefly describe the instruction sets found in many of the

microprocessors used today. The ISA of a processor can be described using 5 categories:

Operand Storage in the CPU

Where are the operands kept other than in memory?

Number of explicit named operands

How many operands are named in a typical instruction.

Operand location

Can any ALU instruction operand be located in memory? Or must all operands be kept internally in the CPU?

Operations

What operations are provided in the ISA.

Type and size of operands

What is the type and size of each operand and how is it specified?

Of all the above the most distinguishing factor is the first.

The 3 most common types of ISAs are:

1. Stack - The operands are implicitly on top of the stack.
2. Accumulator - One operand is implicitly the accumulator.
3. General Purpose Register (GPR) - All operands are explicitly mentioned, they are either registers or memory locations.

Let's look at the assembly code of C = A + B;in all 3 architectures:

Stack	**Accumulator**	**GPR**
PUSH A	LOAD A	LOAD R1,A
PUSH B	ADD B	ADD R1,B
ADD	STORE C	STORE R1,C
POP C	-	-

Not all processors can be neatly tagged into one of the above categories. The i8086 has many instructions that use implicit

operands although it has a general register set. The i8051 is another example-it has 4 banks of GPRs but most instructions must have the A register as one of its operands.

Stack

Advantages: Simple Model of expression evaluation (reverse polish). Short instructions.

Disadvantages: A stack can't be randomly accessed This makes it hard to generate efficient code. The stack itself is accessed every operation and becomes a bottleneck.

Accumulator

Advantages: Short instructions.

Disadvantages: The accumulator is only temporary storage so memory traffic is the highest for this approach.

GPR

Advantages: Makes code generation easy. Data can be stored for long periods in registers.

Disadvantages: All operands must be named leading to longer instructions.

Earlier CPUs were of the first 2 types but in the last 15 years all CPUs made are GPR processors. The 2 major reasons are that registers are faster than memory, the more data that can be kept internally in the CPU the faster the program will run. The other reason is that registers are easier for a compiler to use.

Reduced Instruction Set Computer (RISC)

As we mentioned before most modern CPUs are of the GPR (General Purpose Register) type. A few examples of such CPUs are the IBM 360, DEC VAX, Intel 80x86 and Motorola 68xxx. But while these CPUS were clearly better than previous stack and accumulator based CPUs they were still lacking in several areas:

1. Instructions were of varying length from 1 byte to 6-8 bytes. This causes problems with the pre-fetching and pipelining of instructions.

2. ALU (Arithmetic Logical Unit) instructions could have operands that were memory locations. Because the number of cycles it takes to access memory varies so does the whole instruction. This isn't good for compiler writers, pipelining and multiple issue.

3. Most ALU instructions had only 2 operands where one of the operands is also the destination. This means this operand is destroyed during the operation or it must be saved before somewhere.

Thus in the early 80's the idea of RISC was introduced. The SPARC project was started at Berkeley and the MIPS project at Stanford. RISC stands for Reduced Instruction Set Computer. The ISA is composed of instructions that all have exactly the same size, usually 32 bits. Thus they can be pre-fetched and pipelined successfully. All ALU instructions have 3 operands which are only registers. The only memory access is through explicit LOAD/STORE instructions.

Thus C = A + B will be assembled as:

LOAD R1,A LOAD R2,B ADDR3,R1,R2 STORE C,R3
Although it takes 4 instructions we can reuse the values in the registers.

Why is this architecture called RISC? What is reduced about it?

To make all instructions the same length the number of bits that are used for the opcode is reduced. Thus less instructions are provided. The instructions that were thrown out are the less important string and BCD (binary-coded decimal) operations. In fact, now that memory access is restricted there aren't several kinds of MOV or ADD instructions. Thus the older architecture

is called CISC (Complete Instruction Set Computer). RISC architectures are also called LOAD/STORE architectures.

The number of registers in RISC is usually 32 or more. The first RISC CPU the MIPS 2000 has 32 GPRs as opposed to 16 in the 68xxx architecture and 8 in the 80x86 architecture. The only disadvantage of RISC is its code size. Usually more instructions are needed and there is a waste in short instructions (POP, PUSH).

ADRESSING MODES AND INSTRUCTION CYCLE

The operation field of an instruction specifies the operation to be performed. This operation will be executed on some data which is stored in computer registers or the main memory. The way any operand is selected during the program execution is dependent on the addressing mode of the instruction. The purpose of using addressing modes is as follows:

1. To give the programming versatility to the user.

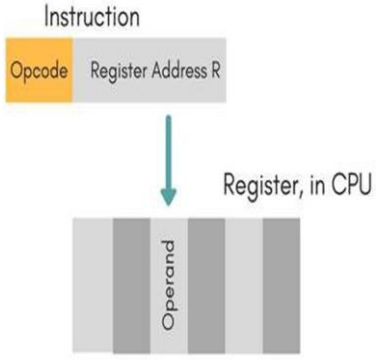
2. To reduce the number of bits in addressing field of instruction.

Types of Addressing Modes:

Immediate Mode

In this mode, the operand is specified in the instruction itself. An immediate mode instruction has an operand field rather than the address field.

For example: ADD 7, which says Add 7 to contents of accumulator. 7 is the operand here.



Register Mode

In this mode the operand is stored in the register and this register is present in CPU. The instruction has the address of the Register where the operand is stored.

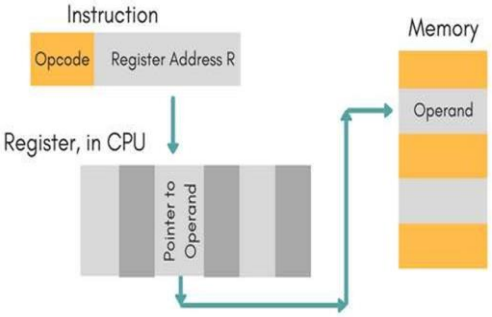
Advantages

- Shorter instructions and faster instruction fetch.
- Faster memory access to the operand(s)

Disadvantages

- Very limited address space
- Using multiple registers helps performance but it complicates the instructions.

Register Indirect Mode

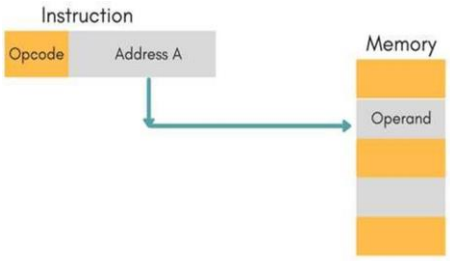


In this mode, the instruction specifies the register whose contents give us the address of operand which is in memory. Thus, the register contains the address of operand rather than the operand itself.

Auto Increment/Decrement Mode

In this the register is incremented or decremented after or before its value is used.

Direct Addressing Mode

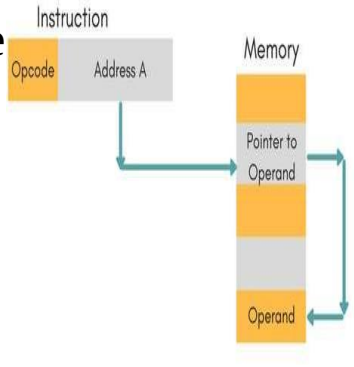


In this mode, effective address of operand is present in instruction itself.

- Single memory reference to access data.
- No additional calculations to find the effective address of the operand.

For Example: ADD R1, 4000 - In this the 4000 is effective address of operand.

NOTE: Effective Address is the location where operand is present.

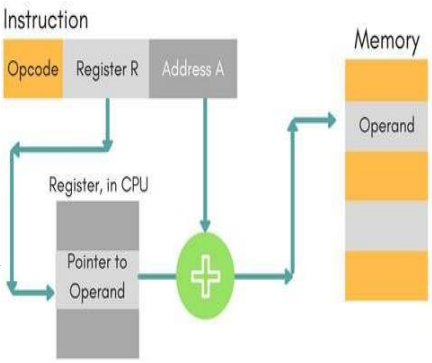


Indirect Addressing Mode

In this, the address field of instruction gives the address where the effective address is stored in memory. This slows down the execution, as this includes multiple memory lookups to find the operand.

Displacement Addressing Mode

In this the contents of the indexed register is added to the Address part of the instruction, to obtain the effective address of operand.



EA = A + (R), In this the address field holds two values, A(which is the base value) and R(that holds the displacement), or vice versa.

Relative Addressing Mode

It is a version of Displacement addressing mode.

In this the contents of PC(Program Counter) is added to address part of instruction to obtain the effective address.

EA = A + (PC), where EA is effective address and PC is program counter. The operand is A cells away from the current cell(the one pointed to by PC)

Base Register Addressing Mode

It is again a version of Displacement addressing mode. This can be defined as EA = A + (R), where A is displacement and R holds pointer to base address.

Stack Addressing Mode

In this mode, operand is at the top of the stack. For example: ADD, this instruction will POP top two items from the stack, add them, and will then PUSH the result to the top of the stack.

INSTRUCTION CYCLE

An instruction cycle, also known as fetch-decode-execute cycle is the basic operational process of a computer. This process is

repeated continuously by CPU from boot up to shut down of computer.

Following are the steps that occur during an instruction cycle:

1. **Fetch the Instruction**

The instruction is fetched from memory address that is stored in PC(Program Counter) and stored in the instruction register IR. At the end of the fetch operation, PC is incremented by 1 and it then points to the next instruction to be executed.

2. **Decode the Instruction**

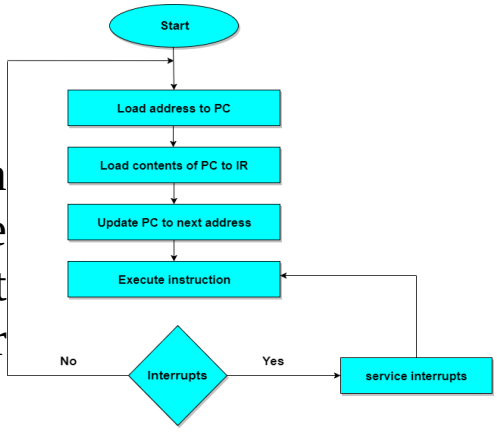
The instruction in the IR is executed by the decoder.

3. **Read the Effective Address**

If the instruction has an indirect address, the effective address is read from the memory. Otherwise operands are directly read in case of immediate operand instruction.

4. **Execute the Instruction**

The Control Unit passes the information in the form of control signals to the functional unit of CPU. The result generated is stored in main memory or sent to an output device.



The cycle is then repeated by fetching the next instruction. Thus in this way the instruction cycle is repeated continuously.

ENCODING AN INSTRUCTION SET

Encoding an instruction into binary representation affects,

- The size of the compiled program
- Implementation of the processor

- The operation is typically specified in one field, called opcode
- The important decision is how to encode the addressing modes with operations.
- The decision depends on the range of addressing modes and the degree of independence between opcodes and modes.
- When encoding the instructions, the number of registers and the number of addressing modes have a significant impact on the size of the instruction.
- The computer architect must balance several competing forces when encoding the instruction set:

The computer architect must balance several competing forces when encoding the instruction set:

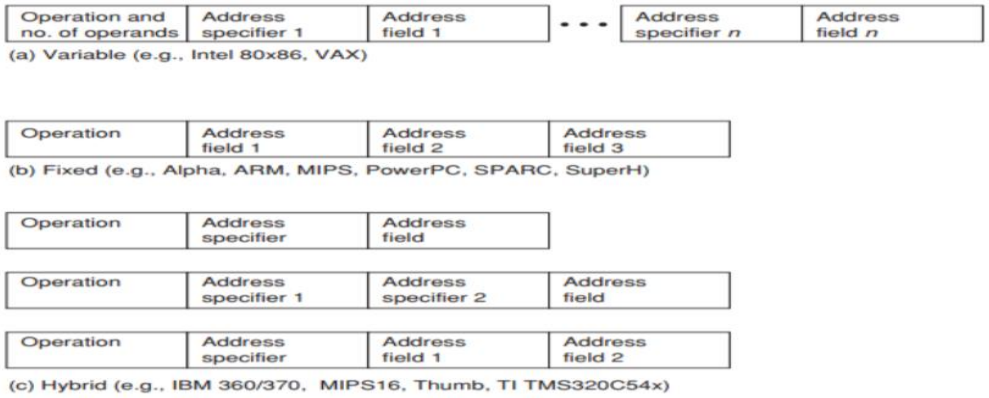
1. The desire to have as many registers and addressing modes as possible.
2. The impact of the size of the register and addressing mode fields on the average instruction size and hence on the average program size.
3. A desire to have instructions encoded into lengths that will be easy to handle in a pipelined implementation.

Three popular choices for encoding the instruction set:

Variable- It allows virtually all addressing modes to be with all operations.

Fixed - It combines the operation and addressing mode into the opcode.

Hybrid - It has multiple formats specified by the opcode



Variable format

- More complex, harder to decode
- More compact, efficient use of memory
- Variable format is best when there are many addressing modes to be with all operations.
- It generally enables the smallest code representation, as no need to include unused fields.

Fixed format

- Simple, easily decoded
- The fixed format always has the same number of operands , with addressing mode specified as part of the opcode
- Fixed encoding will have only single size for all instructions.
- It works best when there are few addressing modes and operations.
- It generally results in the largest code size.

Variable Vs Fixed

- The architect who interested in code size than performance will choose variable encoding.
- The architect who is concerned about performance than code size will choose fixed encoding.

LOGIC DESIGN CONVENTIONS

Types of logic elements

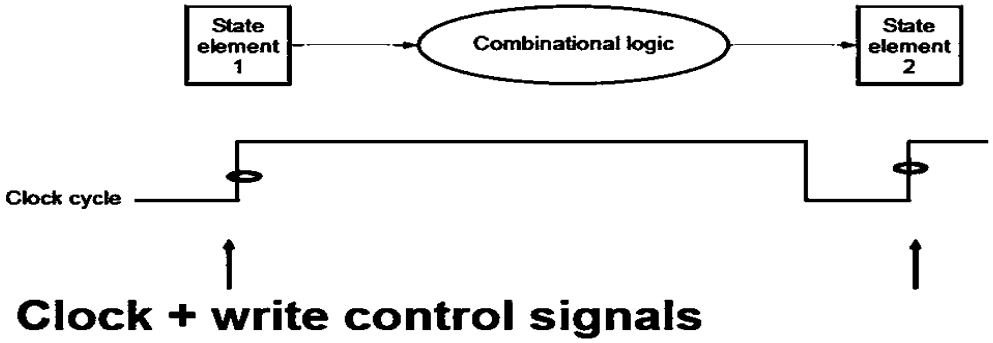
1. Information encoded in binary
    - Low voltage = 0, High voltage = 1
    - One wire per bit
    - Multi-bit data encoded on multi-wire buses
2. Combinational element
    - Operate on data
    - Output is a function of input
    - Output only depends on the current input
    - Uses for ALU, multiplier, and other datapath
3. State (sequential) elements
    - Store information
    - State element to store the states
    - Output depends on current inputs and current states

CLOCKING METHODOLOGY

- Defines when signals can be read and when they can be written
- Mainstream: An edge triggered methodology
- Determine when data is valid and stable relative to the clock

Typical execution:

- read contents of some state elements,
- send values through some combinational logic
- write results to one or more state elements



BUILDING DATA PATH AND CONTROL IMPLEMENTATION SCHEME

Datapath

- Components of the processor that perform arithmetic operations and holds data.

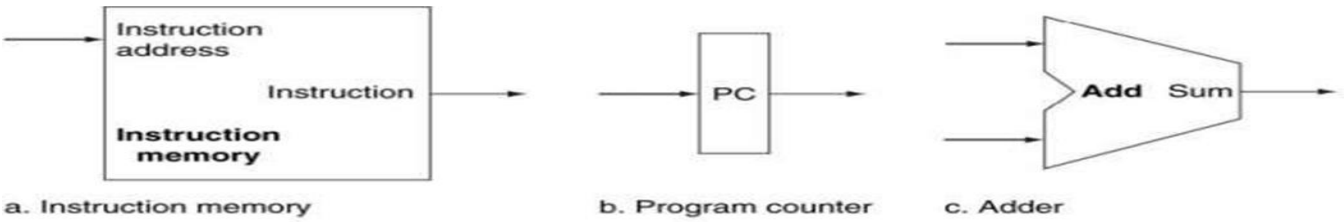
Control

- Components of the processor that commands the datapath, memory, I/O devices according to the instructions of the memory.

Building a Datapath

- Elements that process data and addresses in the CPU - Memories, registers, ALUs.
- MIPS datapath can be built incrementally by considering only a subset of instructions

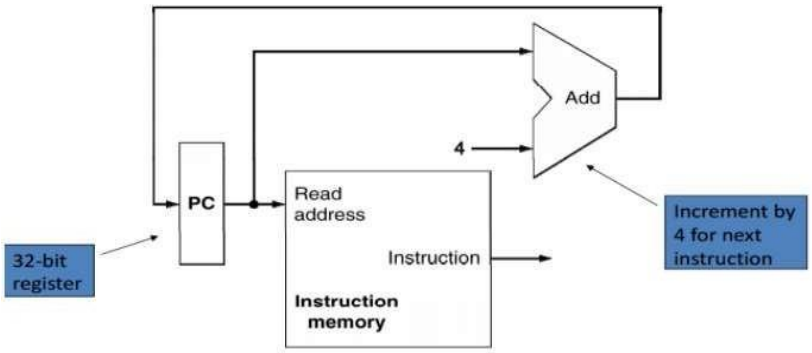
3 main elements are



a. Instruction memory b. Program counter c. Adder

- A memory unit to store instructions of a program and supply instructions given an address.
- Needs to provide only read access (once the program is loaded).
- No control signal is needed

- PC (Program Counter or Instruction address register) is a register that holds the address of the current instruction
- A new value is written to it every clock cycle. No control signal is required to enable write
- Adder to increment the PC to the address of the next instruction
- An ALU permanently wired to do only addition. No extra control signal required



Types of Elements in the Datapath

State element:

•A memory element, i.e., it contains a state

•E.g., program counter, instruction memory

Combinational element:

•Elements that operate on values

•Eg adder ALU E.g. adder, ALU

Elements required by the different classes of instructions

•Arithmetic and logical instructions

•Data transfer instructions

•Branch instructions

R-Format ALU Instructions

•E.g., add $t1, $t2, $t3

•Perform arithmetic/logical operation

•Read two register operands and write register result **Register file:**

•A collection of the registers
•Any register can be read or written by specifying the number of the register
•Contains the register state of the computer

Read from register

•2 inputs to the register file specifying the numbers
•5 bit wide inputs for the 32 registers
•2 outputs from the register file with the read values
•32 bit wide
•For all instructions. No control required.

Write to register file

•1 input to the register file specifying the number 5 bit wide inputs for the 32 registers
•1 input to the register file with the value to be written 32 bit wide
•Only for some instructions. RegWrite control signal.

ALU

•Takes two 32 bit input and produces a 32 bit output
•Also, sets one-bit signal if the results is 0
•The operation done by ALU is controlled by a 4 bit control signal input. This is set according to the instruction .

PIPELINING

Pipelining is an implementation technique whereby multiple instructions are overlapped in execution

Basic concepts

- Pipelining is an implementation technique whereby multiple instructions are overlapped in execution
- Takes advantage of parallelism
- Key implementation technique used to make fast CPUs.

- A pipeline is like an assembly line.

In a computer pipeline, each step in the pipeline completes a part of an instruction. Like the assembly line, different steps are completing different parts of different instructions in parallel. Each of these steps is called a pipe stage or a pipe segment.

- The stages are connected one to the next to form a pipe—instructions enter at one end, progress through the stages, and exit at the other end.
- The throughput of an instruction pipeline is determined by how often an instruction exits the pipeline.
- The time required between moving an instruction one step down the pipeline is a processor cycle.
- If the starting point is a processor that takes 1 (long) clock cycle per instruction, then pipelining decreases the clock cycle time.
- Pipeline for an integer subset of a RISC architecture that consists of load-store word, branch, and integer ALU operations.

Every instruction in this RISC subset can be implemented in at most 5 clock cycles. The 5 clock cycles are as follows.

1.Instruction fetch cycle (IF):

Send the program counter (PC) to memory and fetch the current instruction from memory.

PC=PC+4

2.Instruction decode/register fetch cycle (ID):

- Decode the instruction and read the registers.
- Do the equality test on the registers as they are read, for a possible branch.

- Compute the possible branch target address by adding the sign-extended offset to the incremented PC.
- Decoding is done in parallel with reading registers, which is possible because the register specifiers are at a fixed location in a RISC architecture,known as fixed-field decoding

3.Execution/effective address cycle(EX):

The ALU operates on the operands prepared in the prior cycle, performing one of three functions depending on the instruction type.

Memory reference: The ALU adds the base register and the offset to form the effective address.

Register-Register ALU instruction: The ALU performs the operation specified by the ALU opcode on the values read from the register file

Register-Immediate ALU instruction: The ALU performs the operation specified by the ALU opcode on the first value read from the register file and the sign-extended immediate.

4. Memory access(MEM):

- If the instruction is a load, memory does a read using the effective address computed in the previous cycle.
- If it is a store, then the memory writes the data from the second register read from the register file using the effective address.

5. Write-back cycle (WB):

Register-Register ALU instruction or Load instruction: Write the result into the register file, whether it comes from the memory system (for a load) or from the ALU (for an ALU instruction).

PIPELINED DATA PATH AND CONTROL

The Classic Five-Stage Pipeline for a RISC Processor

Each of the clock cycles from the previous section becomes a pipe stage—a cycle in the pipeline.

Each instruction takes 5 clock cycles to complete, during each clock cycle the hardware will initiate a new instruction and will be executing some part of the five different instructions.

3 observations:

- Use separate instruction and data memories, which implement with separate instruction and data caches.
- The register file is used in the two stages: one for reading in ID and one for writing in WB, need to perform 2 reads and one write every clock cycle.
- Does not deal with PC, To start a new instruction every clock, we must increment and store the PC every clock, and this must be done during the IF stage in preparation for the next instruction.

To ensure that instructions in different stages of the pipeline do not interfere with one another. This separation is done by introducing pipeline registers between successive stages of the pipeline, so that at the end of a clock cycle all the results from a given stage are stored into a register that is used as the input to the next stage on the next clock cycle.

HANDLING DATA HAZARDS & CONTROL HAZARDS

Hazards: Prevent the next instruction in the instruction stream from executing during its designated clock cycle.

Hazards reduce the performance from the ideal speedup gained by pipelining. 3 classes of hazards:

Structural hazards: arise from resource conflicts when the hardware cannot support all possible combinations of instructions simultaneously in overlapped execution.

Data hazards: arise when an instruction depends on the results of a previous instruction in a way that is exposed by the overlapping of instructions in the pipeline.

Control hazards: arise from the pipelining of branches and other instructions that change the PC.

Performance of Pipelines with Stalls

A stall causes the pipeline performance to degrade from the ideal performance. Speedup from pipelining = [1/ (1+ pipeline stall cycles per instruction)] * Pipeline

Structural Hazards

- When a processor is pipelined, the overlapped execution of instructions requires pipelining of functional units and duplication of resources to allow all possible combinations of instructions in the pipeline.
- If some combination of instructions cannot be accommodated because of resource conflicts, the processor is said to have a structural hazard.

Instances:

- When functional unit is not fully pipelined, Then a sequence of instructions using that unpipelined unit cannot proceed at the rate of one per clock cycle.
- when some resource has not been duplicated enough to allow all combinations of instructions in the pipeline to execute.

To Resolve this hazard

- Stall the the pipeline for 1 clock cycle when the data memory access occurs. A stall is commonly called a pipeline

bubble or just bubble, since it floats through the pipeline taking space but carrying no useful work.

Data Hazards

- A major effect of pipelining is to change the relative timing of instructions by overlapping their execution. This overlap introduces data and control hazards.
- Data hazards occur when the pipeline changes the order of read/write accesses to operands so that the order differs from the order seen by sequentially executing instructions on an unpipelined processor.

Minimizing Data Hazard Stalls by Forwarding

The problem solved with a simple hardware technique called forwarding (also called bypassing and sometimes short-circuiting). Forwards works as:

- The ALU result from both the EX/MEM and MEM/WB pipeline registers is always fed back to the ALU inputs.
- If the forwarding hardware detects that the previous ALU operation has written the register corresponding to a source for the current ALU operation, control logic selects the forwarded result as the ALU input rather than the value read from the register file.

Data Hazards Requiring Stalls

- The load instruction has a delay or latency that cannot be eliminated by forwarding alone. Instead, we need to add hardware, called a pipeline interlock, to preserve the correct execution pattern.
- A pipeline interlock detects a hazard and stalls the pipeline until the hazard is cleared.

- This pipeline interlock introduces a stall or bubble. The CPI for the stalled instruction increases by the length of the stall.

Branch Hazards

- Control hazards can cause a greater performance loss for our MIPS pipeline . When a branch is executed, it may or may not change the PC to something other than its current value plus 4.
- If a branch changes the PC to its target address, it is a taken branch; if it falls through, it is not taken, or untaken.

Reducing Pipeline Branch Penalties

- Simplest scheme to handle branches is to freeze or flush the pipeline, holding or deleting any instructions after the branch until the branch destination is known.
- A higher-performance, and only slightly more complex, scheme is to treat every branch as not taken, simply allowing the hardware to continue as if the branch were not executed. The complexity of this scheme arises from having to know when the state might be changed by an instruction and how to "back out" such a change.
- In simple five-stage pipeline, this predicted-not-taken or predicted untaken scheme is implemented by continuing to fetch instructions as if the branch were a normal instruction.
  - The pipeline looks as if nothing out of the ordinary is happening.
  - If the branch is taken, however, we need to turn the fetched instruction into a no-op and restart the fetch at the target address.
- An alternative scheme is to treat every branch as taken. As soon as the branch is decoded and the target address is

computed, we assume the branch to be taken and begin fetching and executing at the target Performance of Branch Schemes

- Pipeline speedup = Pipeline depth / [1+ Branch frequency × Branch penalty]
- The branch frequency and branch penalty can have a component from both unconditional and conditional branches.

Input Output Interface

The method used for transferring information between external I/O devices and internal storage is known as the I/O interface. We use special communication links to interface the CPU with the peripherals connected to any computer system. We use these communication links to resolve the differences between the CPU and peripheral.

The Input-Output Interface is an essential component in computer systems. It enables communication between the CPU and various external devices. These external devices are peripherals like keyboards, monitors, printers, and storage devices such as hard drives, network interfaces, etc.

Modes of Transfer

We store the binary information received through an external device in the memory unit. The information transferred from the CPU to external devices originates from the memory unit. Although the CPU processes the data, the target and source are always the memory unit. We can transfer this information using three different modes of transfer.

1. **Programmed I/O**
2. **Interrupt- initiated I/O**
3. **Direct memory access(DMA)**

1. Programmed I/O

Programmed I/O uses the I/O instructions written in the computer program. The instructions in the program initiate every data item transfer. Usually, the data transfer is from a memory and CPU register. This case requires constant monitoring by the peripheral device's CPU.

Advantages:

- Programmed I/O is simple to implement.
- It requires very little hardware support.
- CPU checks status bits periodically.

Disadvantages:

- The processor has to wait for a long time for the I/O module to be ready for either transmission or reception of data.
- The performance of the entire system is severely degraded.

2. Interrupt-initiated I/O

In the above section, we saw that the CPU is kept busy unnecessarily. We can avoid this situation by using an interrupt-driven method for data transfer. The interrupt facilities and special commands inform the interface for issuing an interrupt request signal as soon as the data is available from any device. In the meantime, the CPU can execute other programs, and the interface will keep monitoring the i/O device. Whenever it determines that the device is ready for transferring data interface initiates an interrupt request signal to the CPU. As soon as the CPU detects an external interrupt signal, it stops the program it was already executing, branches to the service program to process the I/O transfer, and returns to the program it was initially running.

Working of CPU in terms of interrupts:

- CPU issues read command.
- It starts executing other programs.
- Check for interruptions at the end of each instruction cycle.
- On interruptions:-

- Process interrupt by fetching data and storing it.
- See operation system notes.

- Starts working on the program it was executing.

Advantages:

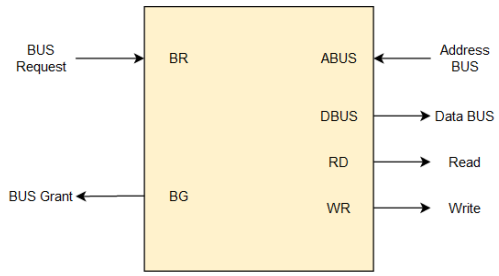
- It is faster and more efficient than Programmed I/O.
- It requires very little hardware support.
- CPU does not check status bits periodically.

Disadvantages:

- It can be tricky to implement if using a low-level language.
- It can be tough to get various pieces of work well together.
- The hardware manufacturer / OS maker usually implements it, e.g., Microsoft.

3. Direct Memory Access (DMA)

The data transfer between any fast storage media like a memory unit and a magnetic disk gets limited with the speed of the CPU. Thus it will be best to allow the peripherals to directly communicate with the storage using the memory buses by removing the intervention of the CPU. This mode of transfer of data technique is known as Direct Memory Access (DMA). During Direct Memory Access, the CPU is idle and has no control over the memory buses. The DMA controller takes over the buses and directly manages data transfer between the memory unit and I/O devices.



CPU Bus Signal for DMA transfer

Bus Request - We use bus requests in the DMA controller to ask the CPU to relinquish the control buses.

Bus Grant - CPU activates bus grant to inform the DMA controller that DMA can take control of the control buses. Once the control is taken, it can transfer data in many ways.

Types of DMA transfer using DMA controller:

- **Burst Transfer:** In this transfer, DMA will return the bus control after the complete data transfer. A register is used as a byte count, which decrements for every byte transfer, and once it becomes zero, the DMA Controller will release the control bus. When the DMA Controller operates in burst mode, the CPU is halted for the duration of the data transfer.
- **Cyclic Stealing:** It is an alternative method for data transfer in which the DMA controller will transfer one word at a time. After that, it will return the control of the buses to the CPU. The CPU operation is only delayed for one memory cycle to allow the data transfer to "steal" one memory cycle.

Advantages

- It is faster in data transfer without the involvement of the CPU.
- It improves overall system performance and reduces CPU workload.
- It deals with large data transfers, such as multimedia and files.

Disadvantages

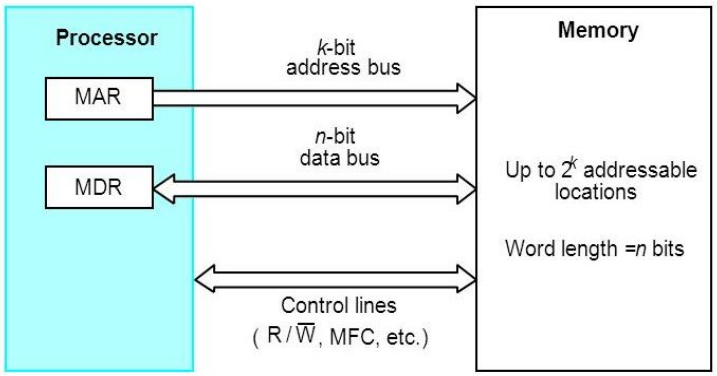
- It is costly and complex hardware.
- It has limited control over the data transfer process.
- Risk of data conflicts between CPU and DMA.

SOME BASIC CONCEPTS

Maximum size of the Main Memory that can be used in any computer is determined by addressing scheme.

- byte-addressable
- CPU-Main Memory Connection

If MAR is k bits long and MDR is n bits long, then the memory may contain upto 2K addressable locations and the n-bits of data are transferred between the memory and processor.



- This transfer takes place over the processor bus.
- The processor bus has,
  - Address Line Data Line
  - Control Line (R/W, MFC – Memory Function Completed)
- The control line is used for co-ordinating data transfer.
- The processor reads the data from the memory by loading the address of the required memory location into MAR and setting the R/W line to 1.
- The memory responds by placing the data from the addressed location onto the data lines and confirms this action by asserting MFC signal

- Upon receipt of MFC signal, the processor loads the data onto the data lines into MDR register.
- The processor writes the data into the memory location by loading the address of this location into MAR and loading the data into MDR sets the R/W line to 0.

Measures for the speed of a memory:

- Memory access time.
  It is the time that elapses between the initiation of an Operation and the completion of that operation.
- Memory cycle time.
  It is the minimum time delay that required between the initiations of the two successive memory operations.

- An important design issue is to provide a computer system with as large and fast a memory as possible, within a given cost target.

- Several techniques to increase the effective size and speed of the memory:

  - Cache memory (to increase the effective speed).
  - Virtual memory (to increase the effective size).
  - RAM (Random Access Memory):
    - In RAM, if any location that can be accessed for a Read/Write operation in fixed amount of time, it is independent of the location's address.
  - Cache Memory:
    - It is a small, fast memory that is inserted between the larger slower main memory and the processor.
    - It holds the currently active segments of a program and their data.
  - Virtual memory:

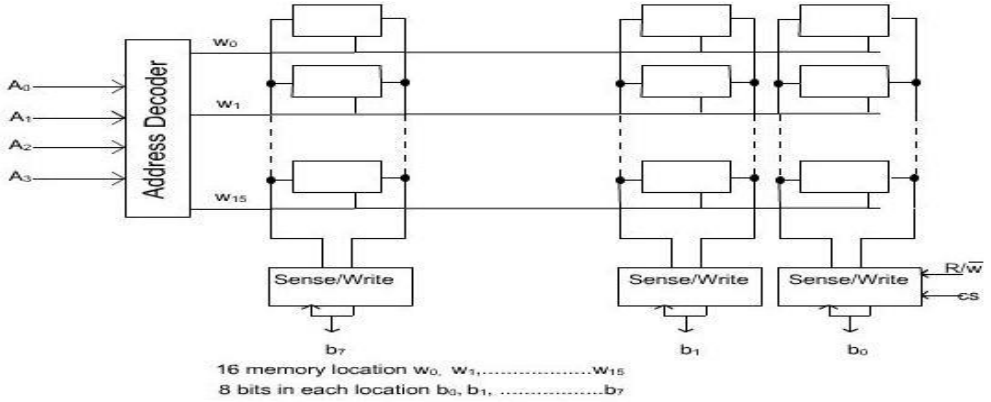
- The address generated by the processor does not directly specify the physical locations in the memory.
- The address generated by the processor is referred to as a virtual / logical address.
- The virtual address space is mapped onto the physical memory where data are actually stored.
- The mapping function is implemented by a special memory control circuit is often called the memory management unit.
- Only the active portion of the address space is mapped into locations in the physical memory.
- The remaining virtual addresses are mapped onto the bulk storage devices used, which are usually magnetic disk.
- As the active portion of the virtual address space changes during program execution, the memory management unit changes the mapping function and transfers the data between disk and memory.
- Thus, during every memory cycle, an address processing mechanism determines whether the addressed in function is in the physical memory unit.
- If it is, then the proper word is accessed and execution proceeds. If it is not, a page of words containing the desired word is transferred from disk to memory.
- This page displaces some page in the memory that is currently inactive.

SEMICONDUCTOR RAM MEMORIES

Internal organization of memory chips

- Each memory cell can hold one bit of information.
- Memory cells are organized in the form of an array.

- One row is one memory word.
- All cells of a row are connected to a common line, known as the “word line”.
- Word line is connected to the address decoder.
- Sense/write circuits are connected to the data
- input/output lines of the memory chip.

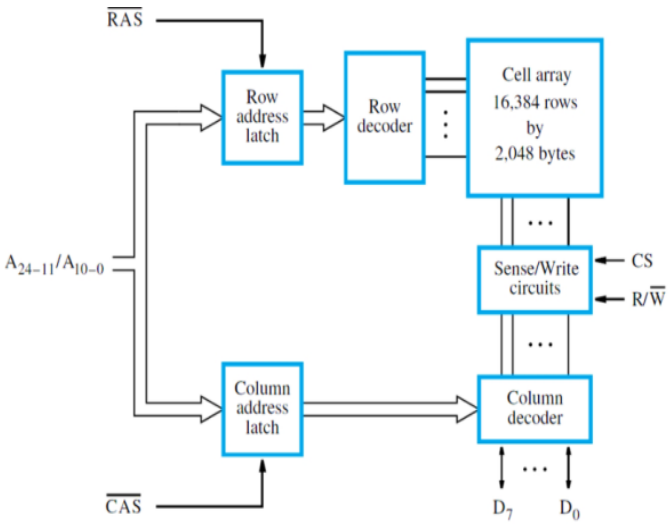


SRAM Cell

- Two transistor inverters are cross connected to implement a basic flip-flop.
- The cell is connected to one word line and two bits lines by transistors T1 and T2
- When word line is at ground level, the transistors are turned off and the latch retains its state
- Read operation: In order to read state of SRAM cell, the word line is activated to close switches T1 and T2. Sense/Write circuits at the bottom monitor the state of b and b’

Asynchronous DRAMs:

- Each row can store 512 bytes. 12 bits to select a row, and 9 bits to select a group in a row. Total of 21 bits.



- First apply the row address; RAS signal latches the row address. Then apply the column address, CAS signal latches the address.
- Timing of the memory unit is controlled by a specialized unit which generates RAS and CAS.
- This is asynchronous DRAM

Types of RAM

➢ ***Static RAMs (SRAMs):***

- Consist of circuits that are capable of retaining their state as long as the power is applied.
- Volatile memories, because their contents are lost when power is interrupted.
- Access times of static RAMs are in the range of few nanoseconds.
- However, the cost is usually high.

➢ ***Dynamic RAMs (DRAMs):***

- Do not retain their state indefinitely.
- Contents must be periodically refreshed.
- Contents may be refreshed while accessing them for reading.

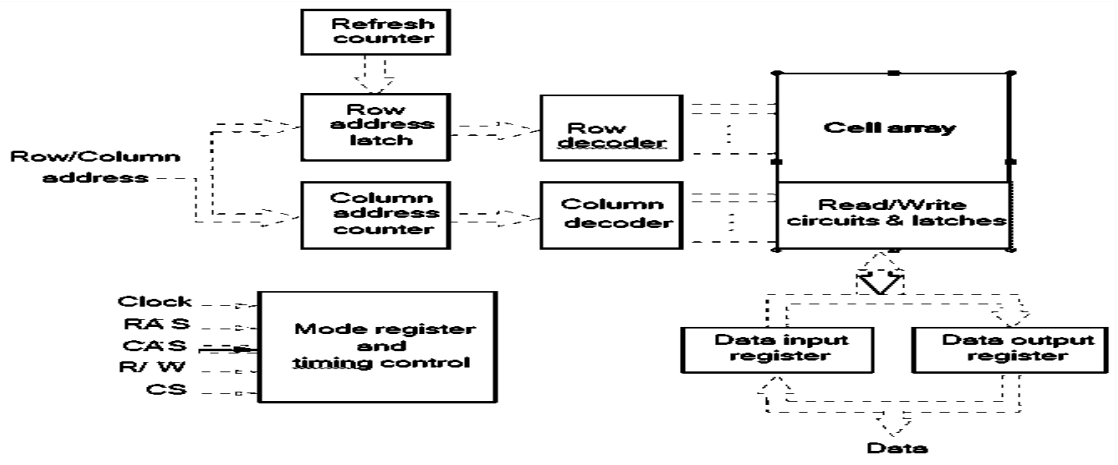
Fast Page Mode

- Suppose if we want to access the consecutive bytes in the selected row.
- This can be done without having to reselect the row.
  - ➢ Add a latch at the output of the sense circuits in each row.
  - ➢ All the latches are loaded when the row is selected.
  - ➢ Different column addresses can be applied to select and place different bytes on the data lines.
- Consecutive sequence of column addresses can be applied under the control signal CAS, without reselecting the row.

- Allows a block of data to be transferred at a much faster rate than random accesses.
- A small collection/group of bytes is usually referred to as a block.

- This transfer capability is referred to as the fast page mode feature.

Synchronous DRAMs



Operation is directly synchronized with processor clock signal.

The outputs of the sense circuits are connected to a latch.

During a Read operation, the contents of the cells in a row are loaded onto the latches.

During a refresh operation, the contents of the cells are refreshed without changing the contents of the latches.

Data held in the latches correspond to the selected columns are transferred to the output.

For a burst mode of operation, successive columns are selected using column address counter and clock. CAS signal need not be generated externally. A new data is placed during raising edge of the clock

Latency, Bandwidth, and DDRSDRAMs

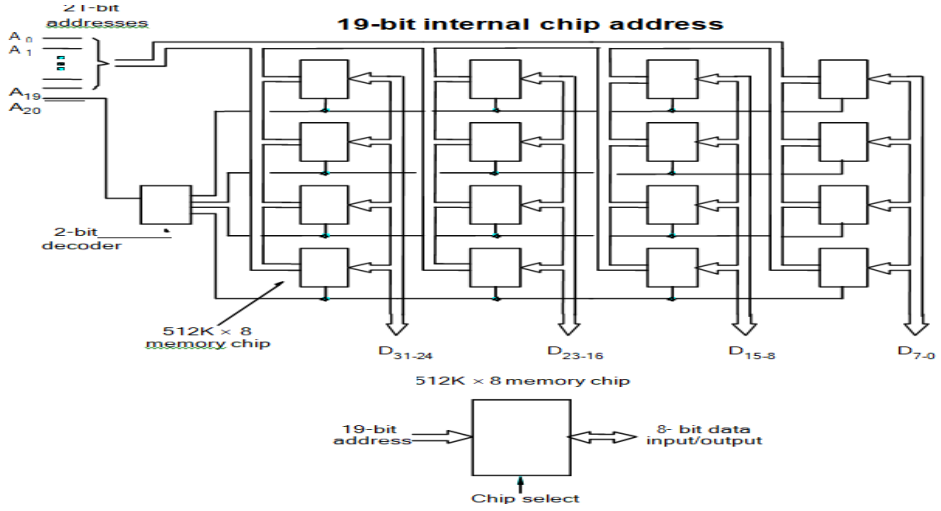
Memory latency is the time it takes to transfer a word of data to or from memory

Memory bandwidth is the number of bits or bytes that can be transferred in one second.

DDRSDRAMs (Double Data rate Synchronous Dynamic Random Access Memory)

Cell array is organized in two banks

Static memories



Implement a memory unit of 2M words of 32 bits each

- Use 512x8 static memory chips.
- Each column consists of 4 chips. Each chip implements one byte position.
- Each chip implements one byte position
- A chip is selected by setting its chip select control line to 1.
- Selected chip places its data on the data output line, outputs of other chips are in high impedance state. 21 bits to address a 32-bit word.
- High order 2 bits are needed to select the row, by activating the four Chip Select signals.
- 19 bits are used to access specific byte locations inside the selected chip

Dynamic memories

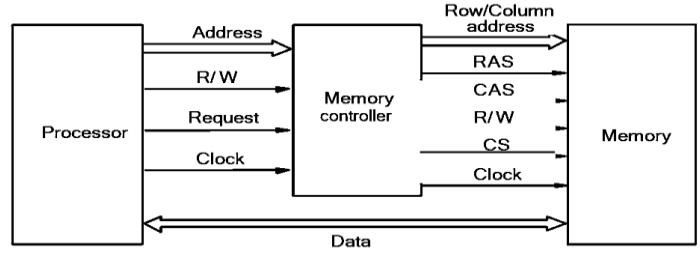
- Large dynamic memory systems can be implemented using DRAM chips in a similar way to static memory systems.
- Placing large memory systems directly on the motherboard will occupy a large amount of space.
- Also, this arrangement is inflexible since the memory system cannot be expanded easily.
- Packaging considerations have led to the development of larger memory units known as SIMMs (Single In-line Memory Modules) and DIMMs (Dual In-line Memory Modules).
- Memory modules are an assembly of memory chips on a small board that plugs vertically onto a single socket on the motherboard.
- Occupy less space on the motherboard.
- Allows for easy expansion by replacement.

Memory controller

- Address is divided into two parts:
  - ➢ High-order address bits select a row in the array.
  - ➢ They are provided first, and latched using RAS signal.
  - ➢ Low-order address bits select a column in the row.
  - ➢ They are provided later, and latched using CAS signal.

- However, a processor issues all address bits at the same time.

- In order to achieve the multiplexing, memory controller circuit is inserted between the processor and memory.



READ-ONLY MEMORIES (ROMS)

- SRAM and SDRAM chips are volatile: Lose the contents when the power is turned off.
- Many applications need memory devices to retain contents after the power is turned off.
  - For example, computer is turned on; the operating system must be loaded from the disk into the memory.
  - Store instructions which would load the OS from the disk.
  - Need to store these instructions so that they will not be lost after the power is turned off.
  - We need to store the instructions into a non-volatile memory.
  - Non-volatile memory is read in the same manner as volatile memory.
  - Separate writing process is needed to place information in this memory.
  - Normal operation involves only reading of data, this type of memory is called Read-Only memory (ROM).

ROM stands for Read-only Memory. It is a type of memory that does not lose its contents when the power is turned off. For this reason, ROM is also called non-volatile memory.

Because ROMs are deployed in such a wide variety of applications, there are different types of ROMs suited to different applications across the industry.

Different Types of ROM

1. PROM (Programmable ROM)
2. EPROM (Erasable Programmable ROM)
3. EEPROM (Electrically Erasable Programmable ROM)
4. Flash EPROM
5. Mask ROM

1. PROM (programmable ROM) and OTP

- PROM refers to the kind of ROM that the user can burn information into.
- In other words, PROM is a user-programmable memory.
- For every bit of the PROM, there exists a fuse. PROM is programmed by blowing the fuses.
- If the information burned into PROM is wrong, that PROM must be discarded since its internal fuses are blown permanently.
- For this reason, PROM is also referred to as OTP (One Time Programmable).
- Programming ROM also called burning ROM, requires special equipment called a ROM burner or ROM programmer.

2. EPROM (erasable programmable ROM) and UV-EPROM

- EPROM was invented to allow making changes in the contents of PROM after it is burned.
- In EPROM, one can program the memory chip and erase it thousands of times.
- This is especially necessary during the development of the prototype of a microprocessor-based project.
- A widely used EPROM is called UV-EPROM, where UV stands for ultraviolet.
- The only problem with UV-EPROM is that erasing its contents can take up to 20 minutes.
- All UV-EPROM chips have a window through which the programmer can shine ultraviolet (UV) radiation to erase the chip's contents. For this reason, EPROM is also referred to as UV-erasable EPROM or simply UV-EPROM.
- To program a UV-EPROM chip, the following steps must be taken:
  - Its contents must be erased.
  - To erase a chip, remove it from its socket on the system board and place it in EPROM erasure

equipment to expose it to UV radiation for 5-20 minutes.

- Program the chip. To program a UV-EPROM chip, place it in the ROM burner (programmer).
- To burn code or data into EPROM, the ROM burner uses 12.5 volts or higher, depending on the EPROM type.
- This voltage is referred to as Vpp in the UV-EPROM datasheet.
- Place the chip back into its socket on the system board.
- As can be seen from the above steps, not only is there an EPROM programmer (burner), but there is also separate EPROM erasure equipment.
- The main problem, and indeed the major disadvantage of UV-EPROM, is that it cannot be erased and programmed while it is in the system board.
- To provide a solution to this problem, EEPROM was invented.

3. EEPROM (electrically erasable programmable ROM)

- EEPROM has several advantages over EPROM, such as the fact that its method of erasure is electrical and therefore instant as opposed to the 20-minute erasure time required for UV-EPROM.
- In addition, in EEPROM one can select which byte to be erased, in contrast to UV-EPROM, in which the entire contents of ROM are erased.
- However, the main advantage of EEPROM is that one can program and erase its contents while it is still in the system board.
- It does not require the physical removal of the memory chip from its socket.
- In other words, unlike UV-EPROM, EEPROM does not require an external erasure and programming device.
- To utilize EEPROM fully, the designer must incorporate the circuitry to program the EEPROM into the system board.

- In general, the cost per bit for EEPROM is much higher than for UV-EPROM.

4. Flash Memory EPROM

- Since the early 1990s, Flash EPROM has become a popular user-programmable memory chip and for good reasons.
- First, the erasure of the entire contents takes less than a second, or one might say in a flash, hence its name, Flash memory.
- In addition, the erasure method is electrical, and for this reason, it is sometimes referred to as Flash EEPROM.
- To avoid confusion, it is commonly called Flash memory.
- The major difference between EEPROM and Flash memory is that when Flash memory's contents are erased, the entire device is erased, in contrast to EEPROM, where one can erase the desired byte.
- Although in many Flash memories recently made available the contents are divided into blocks and the erasure can be done block by block, unlike EEPROM, Flash memory has no byte erasure option.
- Because Flash memory can be programmed while it is in its socket on the system board, it is widely used to upgrade the BIOS ROM of the PC.
- Some designers believe that Flash memory will replace the hard disk as a mass storage medium.
- This would increase the performance of the computer tremendously since Flash memory is semiconductor memory with access time in the range of 100ns compared with disk access time in the range of tens of milliseconds. For this to happen, flash memory's program/erase cycles must become infinite, just like hard disks.
- The program/erase cycle refers to the number of times that a chip can be erased and reprogrammed before it becomes unusable.

- At this time, the program/erase cycle is 100,000 for Flash and EEPROM, 1000 for UV-EPROM, and infinite for RAM and disks.

5. Mask ROM

- Mask ROM refers to a type of ROM in which the contents are programmed by the IC manufacturer.
- In other words, it is not a user-programmable ROM.
- The term mask is used in IC fabrication. Since the process is costly, mask ROM is used when the needed volume is high (hundreds of thousands) and it is absolutely certain that the contents will not change.
- It is common practice to use UV-EPROM or Flash for the development phase of a project, and only after the code/data have been finalized is the mask version of the product ordered.
- The main advantage of mask ROM is its cost since it is significantly cheaper than other kinds of ROM, but if an error is found in the data/code, the entire batch must be thrown away. It must be noted that all ROM memories have 8 bits for data pins; therefore, the organization is x8.

Speed, Size, and Cost

- A big challenge in the design of a computer system is to provide a sufficiently large memory, with a reasonable speed at an affordable cost.

- Static RAM:

  Very fast, but expensive, because a basic SRAM cell has a complex circuit making it impossible to pack a large number of cells onto a single chip.

- Dynamic RAM:

  Simpler basic cell circuit, hence are much less expensive, but significantly slower than SRAMs.

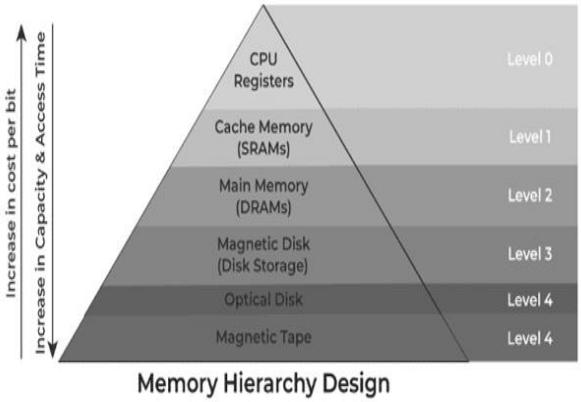
- Magnetic disks:

Storage provided by DRAMs is higher than SRAMs, but is still less than what is necessary.

Secondary storage such as magnetic disks provide a large amount of storage, but is much slower than DRAMs.

Memory Hierarchy

1. Registers



Memory Hierarchy Design

- Registers are small, high-speed memory units located in the CPU.
- They are used to store the most frequently used data and instructions.
- Registers have the fastest access time and the smallest storage capacity, typically ranging from 16 to 64 bits.

2. Cache Memory

- Cache memory is a small, fast memory unit located close to the CPU.
- It stores frequently used data and instructions that have been recently accessed from the main memory.
- Cache memory is designed to minimize the time it takes to access data by providing the CPU with quick access to frequently used data.

3. Main Memory

- Main memory, also known as RAM (Random Access Memory), is the primary memory of a computer system.
- It has a larger storage capacity than cache memory, but it is slower.
- Main memory is used to store data and instructions that are currently in use by the CPU.

Types of Main Memory

- Static RAM:
  Static RAM stores the binary information in flip flops and information remains valid until power is supplied.
  It has a faster access time and is used in implementing cache memory.
- Dynamic RAM:

  It stores the binary information as a charge on the capacitor.

  It requires refreshing circuitry to maintain the charge on the capacitors after a few milliseconds.

  It contains more memory cells per unit area as compared to SRAM.

4. Secondary Storage

- Secondary storage, such as hard disk drives (HDD) and solid-state drives (SSD), is a non-volatile memory unit that has a larger storage capacity than main memory.
- It is used to store data and instructions that are not currently in use by the CPU.
- Secondary storage has the slowest access time and is typically the least expensive type of memory in the memory hierarchy.

5. Magnetic Disk

- Magnetic Disks are simply circular plates that are fabricated with either a metal or a plastic or a magnetized material.
- The Magnetic disks work at a high speed inside the computer and these are frequently used

6. Magnetic Tape

- Magnetic Tape is simply a magnetic recording device that is covered with a plastic film.

- It is generally used for the backup of data.
- In the case of a magnetic tape, the access time for a computer is a little slower and therefore, it requires some amount of time for accessing the strip.

Characteristics of Memory Hierarchy

One can infer these characteristics of a Memory Hierarchy Design from the figure given above:

1. Capacity

- It refers to the total volume of data that a system's memory can store.
- The capacity increases moving from the top to the bottom in the Memory Hierarchy.

2. Access Time

- It refers to the time interval present between the request for read/write and the data availability.
- The access time increases as we move from the top to the bottom in the Memory Hierarchy.

3. Performance

- When a computer system was designed earlier without the Memory Hierarchy Design, the gap in speed increased between the given CPU registers and the Main Memory due to a large difference in the system's access time.
- It ultimately resulted in the system's lower performance, and thus, enhancement was required.
- Such a kind of enhancement was introduced in the form of Memory Hierarchy Design, and because of this, the system's performance increased.
- One of the primary ways to increase the performance of a system is minimising how much a memory hierarchy has to be done to manipulate data.

Cache memory

- It is a small, high-speed storage area in a computer.
- The cache is a smaller and faster memory that stores copies of the data from frequently used main memory locations.
- There are various independent caches in a CPU, which store instructions and data.
- The most important use of cache memory is that it is used to reduce the average time to access data from the main memory.
- By storing this information closer to the CPU, cache memory helps speed up the overall processing time.
- Cache memory is much faster than the main memory (RAM).
- When the CPU needs data, it first checks the cache.
- If the data is there, the CPU can access it quickly.
- If not, it must fetch the data from the slower main memory.

Characteristics of Cache Memory

- Cache memory is an extremely fast memory type that acts as a buffer between RAM and the CPU.
- Cache Memory holds frequently requested data and instructions so that they are immediately available to the CPU when needed.
- Cache memory is costlier than main memory or disk memory but more economical than CPU registers.
- Cache Memory is used to speed up and synchronize with a high-speed CPU.

Levels of Memory

- Level 1 or Register: It is a type of memory in which data is stored and accepted that are immediately stored in the CPU.

The most commonly used register is Accumulator, Program counter, Address Register, etc.

- Level 2 or Cache memory: It is the fastest memory that has faster access time where data is temporarily stored for faster access.
- Level 3 or Main Memory: It is the memory on which the computer works currently. It is small in size and once power is off data no longer stays in this memory.
- Level 4 or Secondary Memory: It is external memory that is not as fast as the main memory but data stays permanently in this memory.

Cache Performance

- When the processor needs to read or write a location in the main memory, it first checks for a corresponding entry in the cache.
- If the processor finds that the memory location is in the cache, a **Cache Hit** has occurred and data is read from the cache.
- If the processor does not find the memory location in the cache, a **Cache miss** has occurred. For a cache miss, the cache allocates a new entry and copies in data from the main memory, then the request is fulfilled from the contents of the cache.
- The performance of cache memory is frequently measured in terms of a quantity called Hit ratio.
- Hit Ratio(H) = hit / (hit + miss) = no. of hits/total accesses
- Miss Ratio = miss / (hit + miss) = no. of miss/total accesses = 1 - hit ratio(H)
- We can improve Cache performance using higher cache block size, and higher associativity, reduce miss rate, reduce miss penalty, and reduce the time to hit in the cache.
- If the data is in the cache it is called a Read or Write hit.

Read hit:

- The data is obtained from the cache.

Write hit:

- Cache has a replica of the contents of the main memory.
- Contents of the cache and the main memory may be updated simultaneously. This is the write-through protocol.
- Update the contents of the cache, and mark it as updated by setting a bit known as the dirty bit or modified bit. The contents of the main memory are updated when this block is replaced. This is *write-back or copy-back protocol.*
- If the data is not present in the cache, then a Read miss or Write miss occurs.

Read miss:

- Block of words containing this requested word is transferred from the memory.
- After the block is transferred, the desired word is forwarded to the processor.
- The desired word may also be forwarded to the processor as soon as it is transferred without waiting for the entire block to be transferred. This is called load-through or early-restart.

Write-miss:

- Write-through protocol is used, then the contents of the main memory are updated directly.
- If write-back protocol is used, the block containing the addressed word is first brought into the cache. The desired word is overwritten with new information.

Application of Cache Memory

Primary Cache: A primary cache is always located on the processor chip. This cache is small and its access time is comparable to that of processor registers.

Secondary Cache: Secondary cache is placed between the primary cache and the rest of the memory. It is referred to as

the level 2 (L2) cache. Often, the Level 2 cache is also housed on the processor chip.

Spatial Locality of Reference: Spatial Locality of Reference says that there is a chance that the element will be present in close proximity to the reference point and next time if again searched then more close proximity to the point of reference.

Temporal Locality of Reference: Temporal Locality of Reference uses the Least recently used algorithm will be used. Whenever there is page fault occurs within a word will not only load the word in the main memory but the complete page fault will be loaded because the spatial locality of reference rule says that if you are referring to any word next word will be referred to in its register that's why we load complete page table so the complete block will be loaded.

Advantages

- Cache Memory is faster in comparison to main memory and secondary memory.
- Programs stored by Cache Memory can be executed in less time.
- The data access time of Cache Memory is less than that of the main memory.
- Cache Memory stored data and instructions that are regularly used by the CPU, therefore it increases the performance of the CPU.

Disadvantages

- Cache Memory is costlier than primary memory and secondary memory.
- Data is stored on a temporary basis in Cache Memory.
- Whenever the system is turned off, data and instructions stored in cache memory get destroyed.
- The high cost of cache memory increases the price of the Computer System.

Cache Coherence Problem

- A bit called as “valid bit” is provided for each block.
- If the block contains valid data, then the bit is set to 1, else it is 0.
- Valid bits are set to 0, when the power is just turned on.
- When a block is loaded into the cache for the first time, the valid bit is set to 1.
- Data transfers between main memory and disk occur directly bypassing the cache.
- When the data on a disk changes, the main memory block is also updated.
- However, if the data is also resident in the cache, then the valid bit is set to 0.
- What happens if the data in the disk and main memory changes and the write-back protocol is being used?
- In this case, the data in the cache may also have changed and is indicated by the dirty bit.
- The copies of the data in the cache, and the main memory are different. This is called the cache coherence problem.
- One option is to force a write-back before the main memory is updated from the disk.

Cache Mapping

There are three different types of mapping used for the purpose of cache memory which is as follows:

Direct Mapping

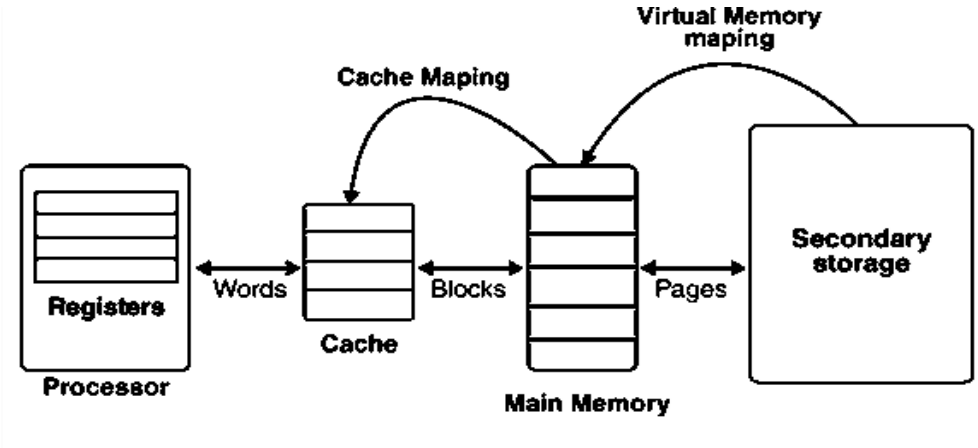
Associative Mapping

Set-Associative Mapping

Process of Cache Mapping

The process of cache mapping helps us define how a certain block that is present in the main memory gets mapped to the memory of a cache in the case of any cache miss.

In simpler words, cache mapping refers to a technique using which we bring the main memory into the cache memory. Here is a diagram that illustrates the actual process of mapping:



The main memory gets divided into multiple partitions of equal size, known as the **frames or blocks**.

The cache memory is actually divided into various partitions of the same sizes as that of the blocks, known as **lines.**

The main memory block is copied simply to the cache during the process of cache mapping, and this block isn't brought at all from the main memory.

Techniques of Cache Mapping

One can perform the process of cache mapping using these three techniques given as follows:

1. K-way Set Associative Mapping

2. Direct Mapping

3. Fully Associative Mapping

1. Direct Mapping

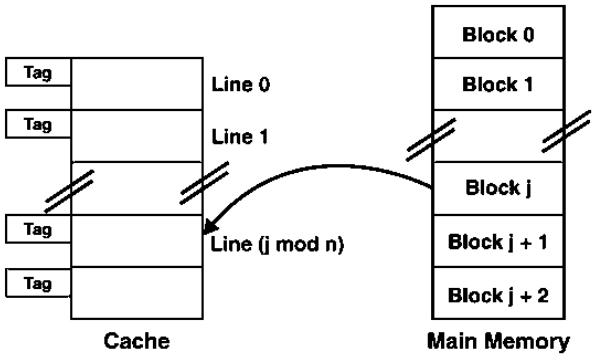
In the case of direct mapping, a certain block of the main memory would be able to map a cache only up to a certain line of the cache. The total line numbers of cache to which any distinct block can map are given by the following:

Cache line number = (Address of the Main Memory Block) Modulo (Total number of lines in Cache)

For example,

Let us consider that particular cache memory is divided into a total of ‘n’ number of lines.

Then, the block ‘j’ of the main memory would be able to map to line number only of the cache (j mod n).



The Need for Replacement Algorithm

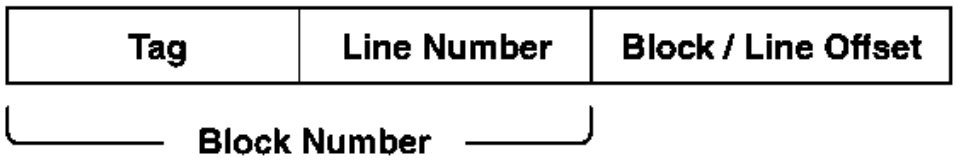
In the case of direct mapping,There is no requirement for a replacement algorithm.

It is because the block of the main memory would be able to map to a certain line of the cache only.

Thus, the incoming (new) block always happens to replace the block that already exists, if any, in this certain line.

Division of Physical Address

In the case of direct mapping, the division of the physical address occurs as follows:



Division of Physical Address in Direct Mapping

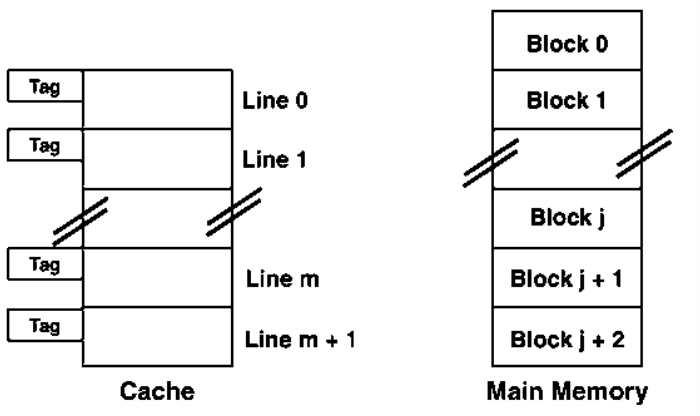
2. Fully Associative Mapping

In the case of fully associative mappingThe main memory block is capable of mapping to any given line of the cache that's available freely at that particular moment.

It helps us make a fully associative mapping comparatively more flexible than direct mapping.

For Example

Let us consider the scenario given as follows:



Here, we can see that,

Every single line of cache is available freely.

Thus, any main memory block can map to a line of the cache.

In case all the cache lines are occupied, one of the blocks that exists already needs to be replaced.

The Need for Replacement Algorithm

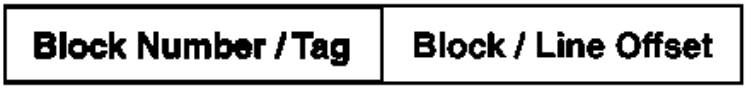
In the case of fully associative mapping,The replacement algorithm is always required.

The replacement algorithm suggests a block that is to be replaced whenever all the cache lines happen to be occupied.

So, replacement algorithms such as LRU Algorithm, FCFS Algorithm, etc., are employed.

Division of Physical Address

In the case of fully associative mapping, the division of the physical address occurs as follows:



Division of Physical Address in Fully Associative Mapping

3. K-way Set Associative Mapping

In the case of k-way set associative mapping,

The grouping of the cache lines occurs into various sets where all the sets consist of k number of lines.

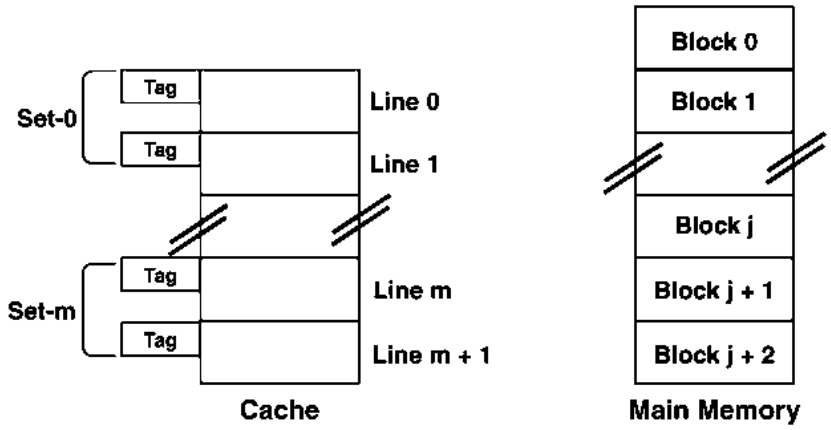
Any given main memory block can map only to a particular cache set.

However, within that very set, the block of memory can map any cache line that is freely available.

The cache set to which a certain main memory block can map is basically given as follows:

Cache set number = (Block Address of the Main Memory) Modulo (Total Number of sets present in the Cache)

For Example:Let us consider the example given as follows of a two-way set-associative mapping:



2-Way Set Associative Mapping

In this case,

k = 2 would suggest that every set consists of two cache lines.

Since the cache consists of 6 lines, the total number of sets that are present in the cache = 6 / 2 = 3 sets.

The block ‘j’ of the main memory is capable of mapping to the set number only (j mod 3) of the cache.

Here, within this very set, the block ‘j’ is capable of mapping to any cache line that is freely available at that moment.

In case all the available cache lines happen to be occupied, then one of the blocks that already exist needs to be replaced.

The Need for Replacement Algorithm

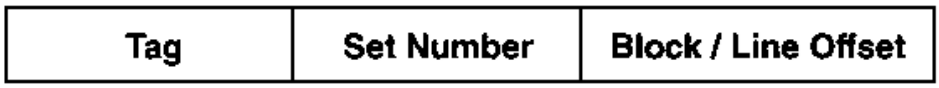
In the case of k-way set associative mapping, The k-way set associative mapping refers to a combination of the direct mapping as well as the fully associative mapping.

It makes use of the fully associative mapping that exists within each set.

Therefore, the k-way set associative mapping needs a certain type of replacement algorithm.

Division of Physical Address

In the case of fully k-way set mapping, the division of the physical address occurs as follows:



Division of Physical Address in K-way Set Associative Mapping

Special Cases: In case k = 1, the k-way set associative mapping would become direct mapping. Thus,

Direct Mapping = one-way set associative mapping

In the case of k = The total number of lines present in the cache, then the k-way set associative mapping would become fully associative mapping.

OTHER PERFORMANCEENHANCEMENTS

Write buffer

Write-through:

- Each write operation involves writing to the main memory.
- If the processor has to wait for the write operation to be complete, it slows down the processor.
- Processor does not depend on the results of the write operation.
- Write buffer can be included for temporary storage of write requests.
- Processor places each write request into the buffer and continues execution.
- If a subsequent Read request references data which is still in the write buffer, then this data is referenced in the write buffer.

Write-back:

- Block is written back to the main memory when it is replaced.
- If the processor waits for this write to complete, before reading the new block, it is slowed down.
- Fast write buffer can hold the block to be written, and the new block can be read first.

Prefetching

- New data are brought into the processor when they are first needed.
- Processor has to wait before the data transfer is complete.
- Prefetch the data into the cache before they are actually needed, or a before a Read miss occurs.
- Prefetching can be accomplished through software by including a special instruction in the machine language of the processor.

 Inclusion of prefetch instructions increases the length of the programs.

- Prefetching can also be accomplished using hardware:

 Circuitry that attempts to discover patterns in memory references and then prefetches according to this pattern.

Lockup-Free Cache

- Prefetching scheme does not work if it stops other accesses to the cache until the prefetch is completed.
- A cache of this type is said to be “locked” while it services a miss.
- Cache structure which supports multiple outstanding misses is called a lockup free cache.

- Since only one miss can be serviced at a time, a lockup free cache must include circuits that keep track of all the outstanding misses.
- Special registers may hold the necessary information about these misses.

Replacement Algorithm

Page Replacement Algorithm decides which page to remove, also called swap out when a new page needs to be loaded into the main memory. Page Replacement happens when a requested page is not present in the main memory and the available space is not sufficient for allocation to the requested page.

When the page that was selected for replacement was paged out, and referenced again, it has to read in from disk, and this requires for I/O completion. This process determines the quality of the page replacement algorithm: the lesser the time waiting for page-ins, the better is the algorithm.

A page replacement algorithm tries to select which pages should be replaced so as to minimize the total number of page misses. There are many different page replacement algorithms. These algorithms are evaluated by running them on a particular string of memory reference and computing the number of page faults. The fewer is the page faults the better is the algorithm for that situation.

If a process requests for page and that page is found in the main memory then it is called ***page hit*** *, otherwise* ***page miss*** *or* ***page fault*** *.*

Replacement Algorithms :

- First In First Out (FIFO)

- Least Recently Used (LRU)
- Optimal Page Replacement

First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the OS maintains a queue that keeps track of all the pages in memory, with the oldest page at the front and the most recent page at the back.

When there is a need for page replacement, the FIFO algorithm, swaps out the page at the front of the queue, that is the page which has been in the memory for the longest time.

For Example:

Consider the page reference string of size 12: 1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3 with frame size 4(i.e. maximum 4 pages in a frame).

1	2	3	4	5	1	3	1	6	3	2	3
1	1	1	1	5	5	5	5	5	5	2	2
	2	2	2	2	1	1	1	1	1	1	1
		3	3	3	3	3	3	6	6	6	6
			4	4	4	4	4	4	3	3	3
M	M	M	M	M	M	H	H	M	M	M	H

M = Miss
H = Hit

Total Page Fault = 9

Initially, all 4 slots are empty, so when 1, 2, 3, 4 came they are allocated to the empty slots in order of their arrival. This is page fault as 1, 2, 3, 4 are not available in memory.

When 5 comes, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 1.

When 1 comes, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 2.

When 3,1 comes, it is available in the memory, i.e., Page Hit, so no replacement occurs.

When 6 comes, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 3.

When 3 comes, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 4.

When 2 comes, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 5.

When 3 comes, it is available in the memory, i.e., Page Hit, so no replacement occurs.

Page Fault ratio = 9/12 i.e. total miss/total possible cases

Advantages

- Simple and easy to implement.
- Low overhead.

Disadvantages

- Poor performance.
- Doesn't consider the frequency of use or last used time, simply replaces the oldest page.
- Suffers from Belady's Anomaly(i.e. more page faults when we increase the number of page frames).

Least Recently Used (LRU)

Least Recently Used page replacement algorithm keeps track of page usage over a short period of time. It works on the idea that the pages that have been most heavily used in the past are most likely to be used heavily in the future too.

In LRU, whenever page replacement happens, the page which has not been used for the longest amount of time is replaced.

For Example

1	2	3	4	5	1	3	1	6	3	2	3
1	1	1	1	5	5	5	5	5	5	2	2
	2	2	2	2	1	1	1	1	1	1	1
		3	3	3	3	3	3	3	3	3	3
			4	4	4	4	4	6	6	6	6
M	M	M	M	M	M	H	H	M	H	M	H

M = Miss
H = Hit

Total Page Fault = 8

Initially, all 4 slots are empty, so when 1, 2, 3, 4 came they are allocated to the empty slots in order of their arrival. This is page fault as 1, 2, 3, 4 are not available in memory.

When 5 comes, it is not available in memory so page fault occurs and it replaces 1 which is the least recently used page.

When 1 comes, it is not available in memory so page fault occurs and it replaces 2.

When 3,1 comes, it is available in the memory, i.e., Page Hit, so no replacement occurs.

When 6 comes, it is not available in memory so page fault occurs and it replaces 4.

When 3 comes, it is available in the memory, i.e., Page Hit, so no replacement occurs.

When 2 comes, it is not available in memory so page fault occurs and it replaces 5.

When 3 comes, it is available in the memory, i.e., Page Hit, so no replacement occurs.

Page Fault ratio = 8/12

Advantages

- Efficient.
- Doesn't suffer from Belady’s Anomaly.

Disadvantages

- Complex Implementation.
- Expensive.
- Requires hardware support.

Optimal Page Replacement

Optimal Page Replacement algorithm is the best page replacement algorithm as it gives the least number of page faults. It is also known as OPT, clairvoyant replacement algorithm, or Belady’s optimal page replacement policy.

In this algorithm, pages are replaced which would not be used for the longest duration of time in the future, i.e., the pages in the memory which are going to be referred farthest in the future are replaced.

This algorithm was introduced long back and is difficult to implement because it requires future knowledge of the program

behaviour. However, it is possible to implement optimal page replacement on the second run by using the page reference information collected on the first run.

For Example

1	2	3	4	5	1	3	1	6	3	2	3

1	1	1	1	1	1	1	1	6	6	6	6
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	5	5	5	5	5	5	5	5
M	M	M	M	M	H	H	H	M	H	H	H

M = Miss
H = Hit

Total Page Fault = 6

Initially, all 4 slots are empty, so when 1, 2, 3, 4 came they are allocated to the empty slots in order of their arrival. This is page fault as 1, 2, 3, 4 are not available in memory.

When 5 comes, it is not available in memory so page fault occurs and it replaces 4 which is going to be used farthest in the future among 1, 2, 3, 4.

When 1,3,1 comes, they are available in the memory, i.e., Page Hit, so no replacement occurs.

When 6 comes, it is not available in memory so page fault occurs and it replaces 1.

When 3, 2, 3 comes, it is available in the memory, i.e., Page Hit, so no replacement occurs.

Page Fault ratio = 6/12

Advantages

- Easy to Implement.
- Simple data structures are used.
- Highly efficient.

Disadvantages

- Requires future knowledge of the program.
- Time-consuming.